

```
In [1]: # Behzad Ghabaei
# CS 82A - Data Science
# Module 5 Lab 1: Chart It, Then Test It
# module5_lab.ipynb
# Instructor Seno
# 9/29/2026
```

Step 1: Group means for chick feeds

Load chickwts.csv and compute mean weight per feed type, sorted (Tutorial 4 pattern). You should see: six feeds; casein highest, horsebean lowest.

```
In [2]: import seaborn as sns
import scipy
import matplotlib.pyplot as plt
import pandas as pd
df = pd.read_csv("chickwts.csv")
```

This is how we import libraries and load the CSV file.

```
In [3]: df.columns
```

```
Out[3]: Index(['Unnamed: 0', 'weight', 'feed'], dtype='str')
```

```
In [4]: df.groupby("feed")["weight"].mean().sort_values()
```

```
Out[4]: feed
horsebean    160.200000
linseed      218.750000
soybean      246.428571
meatmeal     276.909091
casein       323.583333
sunflower    328.916667
Name: weight, dtype: float64
```

This is how we use the groupby() function to compare the weight mean of each feed. Six feeds listed.

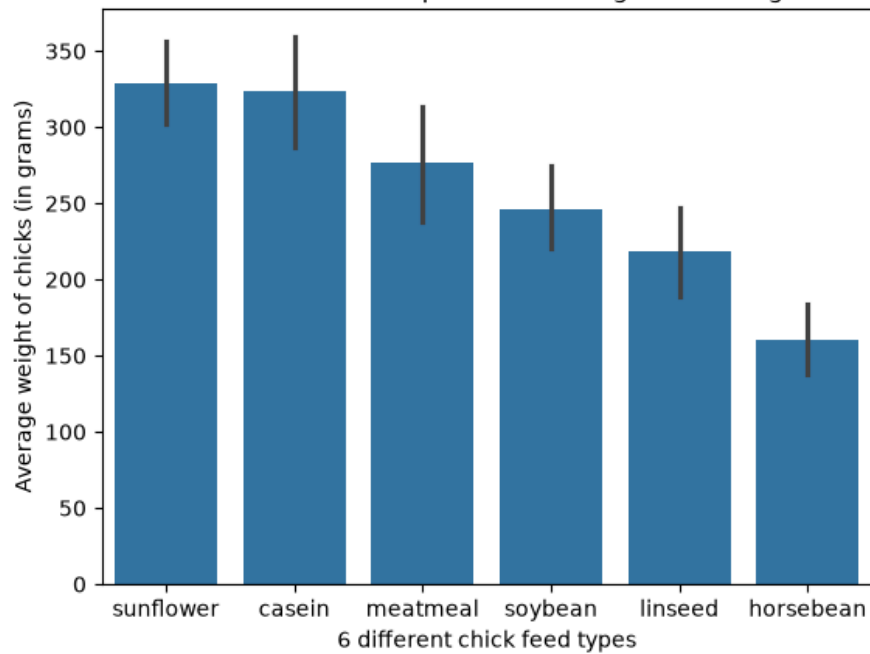
Step 2: Sorted bar chart

Bar-chart mean weight by feed (Tutorial 1: seaborn), fully labeled (Tutorial 2: titles and labels) with a title that states the takeaway and a y-axis in grams. You should see: sorted bars with casein tallest, readable labels.

Adding a takeaway helps the reader to understand the bottom line, this is the title. Sorting the bars requires some additional code. ([geeksforgeeks.org](https://www.geeksforgeeks.org/python/seaborn-sort-bars-in-barplot/) | July 23, 2025 | Sorting Bars in Example 2: Seaborn - Sort Bars in Barplot | <https://www.geeksforgeeks.org/python/seaborn-sort-bars-in-barplot/>)

```
In [5]: sns.barplot(data=df, x="feed", y="weight", order= df.sort_values("weight", ascending = False).feed)
plt.title("Highest to Lowest: Casein and Sunflower produce the highest average chick weight in grams")
plt.xlabel("6 different chick feed types")
plt.ylabel("Average weight of chicks (in grams)")
plt.show()
```

Highest to Lowest: Casein and Sunflower produce the highest average chick weight in grams

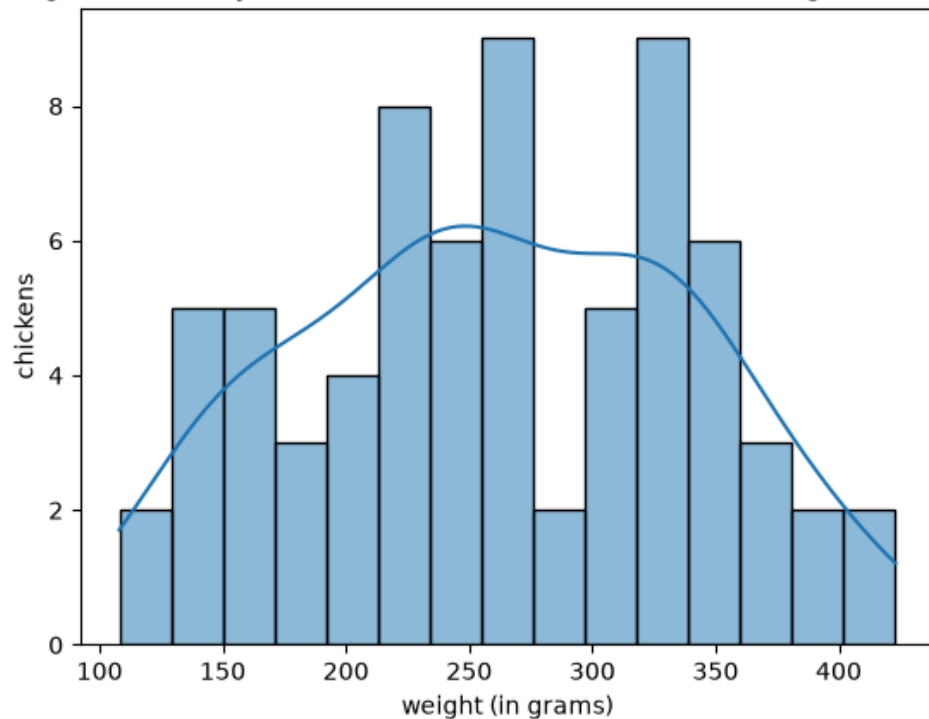


Step 3: Histogram of all weights

Histogram of weight with about 15 bins, labeled. You should see: a roughly symmetric distribution centered near 260 g.

```
In [6]: sns.histplot(data=df, x="weight", bins=15, kde=True)
plt.title("Chick weight forms a symmetric distribution centered at 260 grams from the feed.")
plt.xlabel("weight (in grams)")
plt.ylabel("chickens")
plt.show()
```

Chick weight forms a symmetric distribution centered at 260 grams from the feed.



Step 4: Line chart of museum visitors

Load `museum_visitors.csv` and line-chart visitors over time (Tutorial 3), labeled. In one markdown line, note one visible trend. You should see: a labeled line chart and one observed pattern.

First lets take a look at the new dataset with our tools.

```
In [7]: df_2 = pd.read_csv("museum_visitors.csv")
```

```
In [8]: df_2.columns
```

```
Out[8]: Index(['Date', 'Avila Adobe', 'Firehouse Museum', 'Chinese American Museum',  
             'America Tropical Interpretive Center'],  
            dtype='str')
```

```
In [9]: df_2.info()
```

```
<class 'pandas.DataFrame'>  
RangeIndex: 59 entries, 0 to 58  
Data columns (total 5 columns):  
#   Column                                Non-Null Count  Dtype  
---  -  
0   Date                                59 non-null     str  
1   Avila Adobe                        59 non-null     int64  
2   Firehouse Museum                  59 non-null     int64  
3   Chinese American Museum           59 non-null     int64  
4   America Tropical Interpretive Center 59 non-null     int64  
dtypes: int64(4), str(1)  
memory usage: 2.4 KB
```

```
In [10]: df_2.head()
```

```
Out[10]:
```

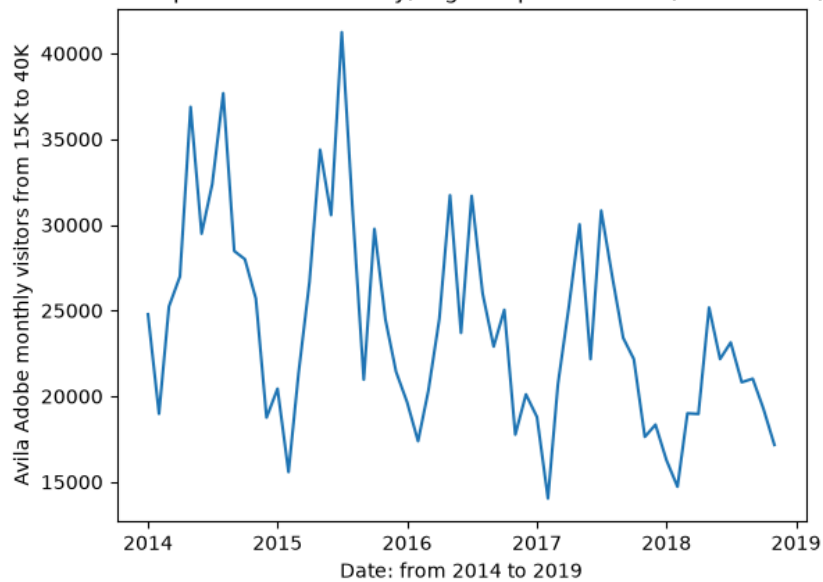
	Date	Avila Adobe	Firehouse Museum	Chinese American Museum	America Tropical Interpretive Center
0	2014-01-01	24778	4486	1581	6602
1	2014-02-01	18976	4172	1785	5029
2	2014-03-01	25231	7082	3229	8129
3	2014-04-01	26989	6756	2129	2824
4	2014-05-01	36883	10858	3676	10694

```
In [11]: df_2["Date"] = pd.to_datetime(df_2["Date"]) #, format="mixed")
```

Lets change the format of time so it can be used in a lineplot.

```
In [12]: sns.lineplot(data=df_2, x="Date", y="Avila Adobe")  
plt.title("2014-2019, museum visits peak twice annually, highest peak in 2015, but lose 15,000 visit")  
plt.xlabel("Date: from 2014 to 2019")  
plt.ylabel("Avila Adobe monthly visitors from 15K to 40K")  
plt.show()
```

2014-2019, museum visits peak twice annually, highest peak in 2015, but lose 15,000 visitors by 2019.

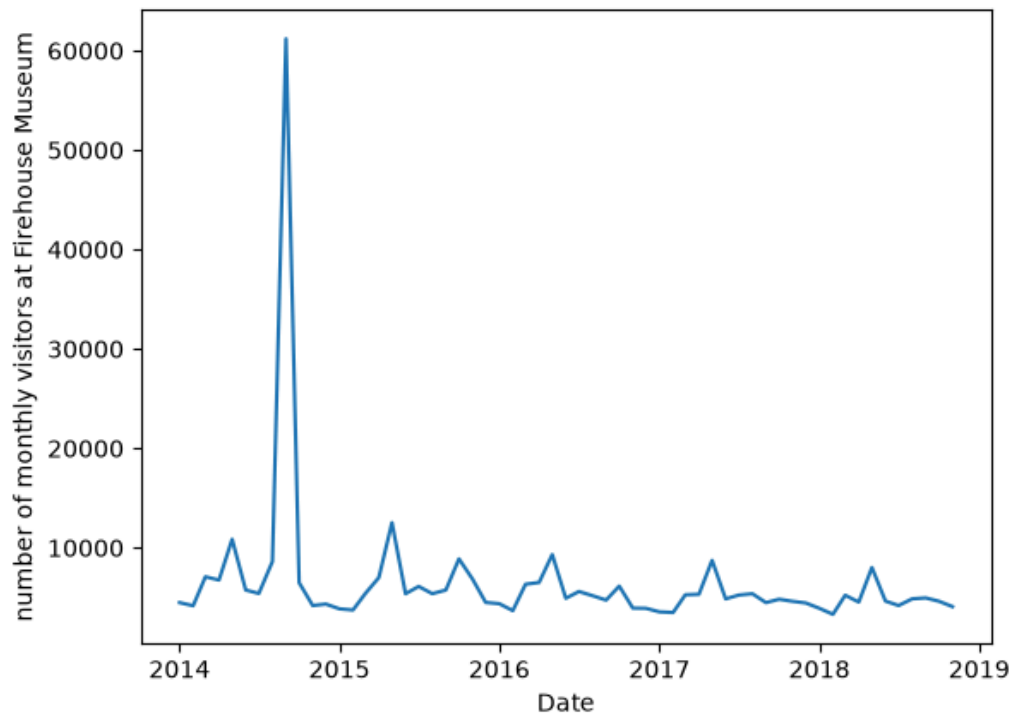


A visible trend: The visits to Avila Adobe look like they occur twice annually, as time elapses the visits at twice a year, dwindle from 35000, to 25000. Museum visits are low at start of each year.

Lets do the other museums also.

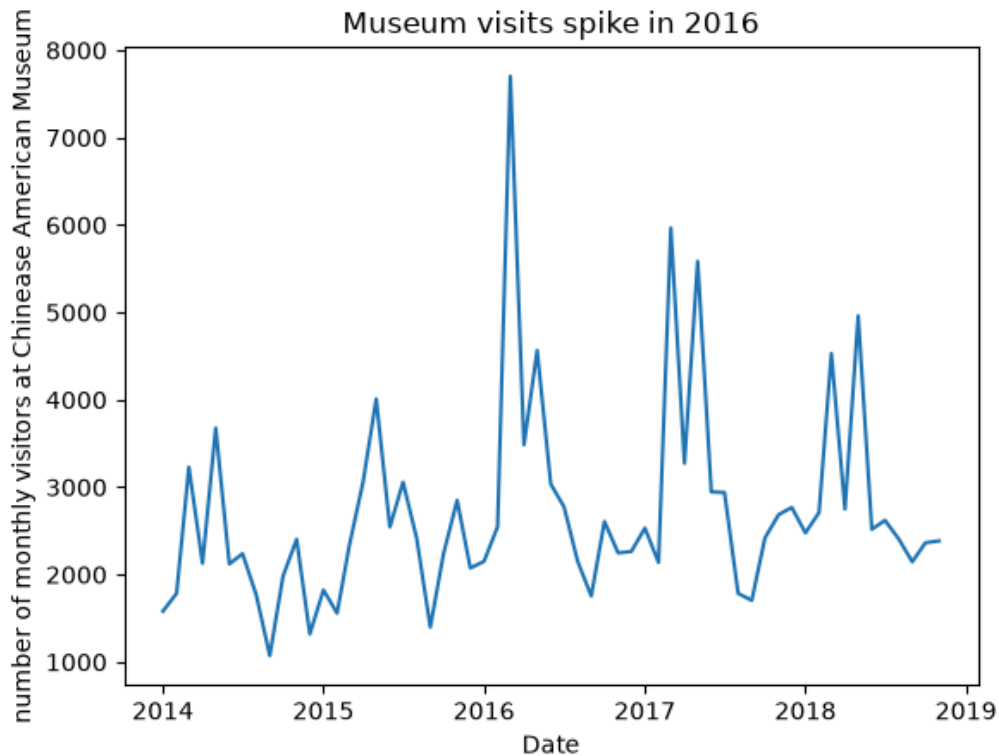
```
In [13]: sns.lineplot(data=df_2, x="Date", y="Firehouse Museum")
plt.title("Museum visits see a boom in Summer of 2014 and sudden bust from 2015 to 2019.")
plt.xlabel("Date")
plt.ylabel("number of monthly visitors at Firehouse Museum")
plt.show()
```

Museum visits see a boom in Summer of 2014 and sudden bust from 2015 to 2019.



A visible trend: The visitors display a spike in 2014 reaching an all time high at 60,000, but from 2015 to 2019 visitations stalemate at 10,000 people.

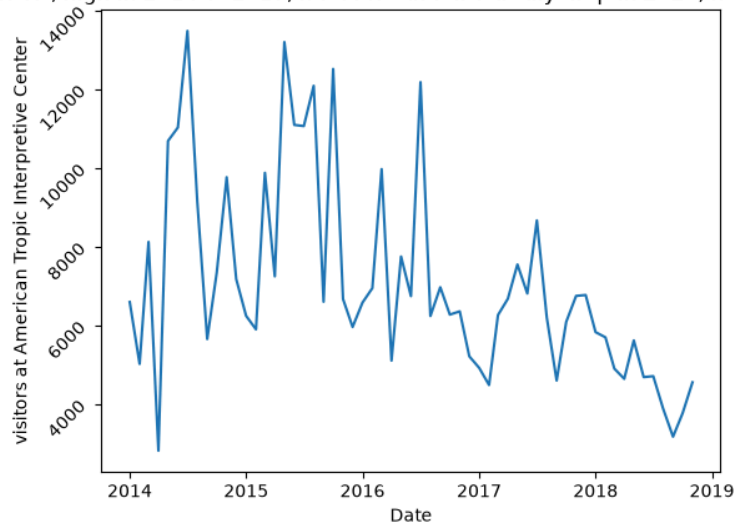
```
In [14]: sns.lineplot(data=df_2, x="Date", y="Chinese American Museum")
plt.title("Museum visits spike in 2016")
plt.xlabel("Date")
plt.ylabel("number of monthly visitors at Chinese American Museum")
plt.show()
```



A visible trend: Chinese American interests spiked in 2016. Steady visits occur twice per month. 2014 to 2019, visitors can reach up to 8000.

```
In [15]: sns.lineplot(data=df_2, x="Date", y="America Tropical Interpretive Center")
plt.title("Mid-year museum visits, high in 2014 to 2017, fluctuate and drastically drop in 2019, from 13,000 to 3,000 visitors.")
plt.xlabel("Date")
plt.ylabel("visitors at American Tropic Interpretive Center")
plt.yticks(rotation=45)
plt.show()
```

Mid-year museum visits, high in 2014 to 2017, fluctuate and drastically drop in 2019, from 13,000 to 3,000 visitors.



A visible trend: Unpredictability and a drastic drop of visits in 2019. The ATIC has seen it's final spikes of interest. Studied from 2014 to 2019.

Step 5: Scatter and heatmap on your sales data

With your Module 4 sales data: scatter qty vs. revenue, and make a correlation heatmap with `sns.heatmap(df.corr(numeric_only=True), annot=True)`. State the strongest off-diagonal correlation in markdown. You should see: the heatmap's annotated numbers match what you found in Lab 4.

The cleaned sales file including revenue is called `sales_clean_revenue.csv` and we can look at any correlation from the heatmap.

```
In [16]: df_3 = pd.read_csv("sales_clean_revenue.csv")
sns.heatmap(df_3.corr(numeric_only=True), annot=True)
```

Out[16]: <Axes: >



The strongest off diagonal correlation is peach color, price vs. revenue, at 0.82 also we make a scatter plot below to confirm the correlation. Notice a cloud across, in a straight line.

```
In [17]: sns.scatterplot(data=df_3, x="price", y="revenue")
plt.title("Total REVENUE increase with PRICE")
plt.xlabel("PRICE in (US dollars)")
plt.ylabel("REVENUE in US dollars")
plt.show()
```



In module 4 the making of a correlation requires that a heatmap square to be between -1 to -0.5 or +0.5 to +1 That means .82 indicates a strong correlation between **price and revenue**. Remember that price * quantity = revenue.

Step 6: State the hypothesis (markdown cell)

You will compare casein-fed and horsebean-fed chicks. Write the null hypothesis in one sentence.

Null hypothesis says there is no effect, no difference between the groups, a random variation, or skeptical position. This means the feed would be no difference between **horsebean** or **casein** The observations made about casein being better and more fattening than horsebean, because the chickens weighed more in grams, is due to random chance. The chicken feed was not the factor that determined the chicken weight, the factor is genetic perhaps or perhaps due to extreme weather conditions. Some chickens experience rain and wind, or have anxiety from surprise attacks from weasels.

Step 7: Run the t-test

Extract the two feed groups' weights and run `ttest_ind` (Tutorial 5 pattern adapted to the chick data). You should see: a large t (about 7 or more) and a p-value far below 0.05.

Comparing means of exactly 2 groups uses a t-test from the `scipy.stats` library. Conventionally evidence against the null hypothesis starts at p below 0.05 also the `groupby()` function can tell us the weight means of each feed type.

```
In [18]: # grouped means before the chart
df.groupby("feed")["weight"].mean().sort_values()
```

```
Out[18]: feed
horsebean    160.200000
linseed      218.750000
soybean       246.428571
meatmeal     276.909091
casein       323.583333
sunflower    328.916667
Name: weight, dtype: float64
```

The function `t-test_ind()` returns t-test results. This is a well known statistical test. It compares the means of two groups. Is the difference statistically significant, or by chance? The output provides information to compare against the confidence interval.

t-value the difference between sample means relative to the variability in samples.

p-value probability that observed difference between sample means occurs by chance.

Reject the null hypothesis if p-value is less than 0.05.

The confidence interval is 95%.

```
In [19]: from scipy.stats import ttest_ind
a = df[df["feed"] == "casein"]["weight"]
b = df[df["feed"] == "horsebean"]["weight"]
ttest_ind(a, b)
```

```
Out[19]: TtestResult(statistic=np.float64(7.019741040397541), pvalue=np.float64(8.254541016953185e-07), df=n
p.float64(20.0))
```

the scope or order of `a, b` makes the **t-value** positive or negative.

p-value: a very small number: 8.254541016953185e-07

large **t-value:** 7.019741040397541

Step 8: The conclusion sentence (markdown cell)

One sentence containing: the difference in grams (effect size), the p-value, your verdict, and one caution (sample size, or that this is one farm's data).

In the `groupby()` function, we were able to see the differences between the averages of chick weight of two types of feed. Horsebean is at 160.20 and casein is at 323.58. A difference of **163.38 grams** between the two types of feed. We look at the p value which is less than 0.001 and we'll reject the null hypothesis. Possible warnings: This was a small dataset, from a single farm. Rain and harsh weather was not recorded. Aggressive animal behavior was not recorded. It is possible that an enormous dataset from different farms, would yield different results, perhaps chicken feed does not matter, however in this type of environment the dataset points to a conclusion that casein builds heavier chickens at least heavier than horsebean which could be due to fat content or better flavor and painless digestion. When the $p < 0.05$ this proves that the difference matters practically, and evidence against the null hypothesis starts.

Step 9: Honest-chart audit (markdown cell)

Pick one of your charts and name two design choices you made for honesty (zero baseline, units, palette, no truncation).

For honesty I tried to prevent labels from overlapping into a smear with `plt.xticks(rotation=45)` `plt.yticks(rotation=45)` allowing more descriptive writing on the

chart.

From lecture 5.1 how bars mislead, The axis starts at 0. Zero baseline starts at zero to ensure the height of the bars are true of the weights to prevent truncated axis. `sns.barplot()` function from seaborn. When Seaborn draws the bars, it calculates the y-axis at 0 so the bars are accurate.

The units on the y-axis are clearly labeled in grams, this clarifies the chick weight is not in pounds. These two genuine design choices will prove to be essential in future honest-chart development.

Behzad Ghabaei - End Of Lab

In []:

